Project Interim Report

On

SIMPLE TEXT EDITOR
(Using Core Java & OOP Concepts)

RU-100-05-00018

Object Oriented Programming with Java

SESSION: 2025-26

Submitted By

AYUSH PRAJAPATI

ERP ID - RU 25 10351

BACHELOR OF COMPUTER APPLICATION WITH AI AND ML IN ASSOCIATION WITH GOOGLE

Under the Guidance of

- HARSH MULTANI SIR

SCHOOL OF INFORMATION AND TECHNOLOGY

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Serial No.	Title	Page Number
1.	Abstract	3
2.	Introduction	4
3.	Scope	6
4.	System Requirements	7
5.	Methodology	8
6.	Flowchart	9
7.	Implementation	10
8.	Gantt Chart	
9.	Conclusions	13
10.	References	14

TABLE OF CONTENT

ABSTRACT

: This project presents a simple console-based text editor developed using Java and Object-Oriented Programming concepts. The main aim is to understand how text can be stored and manipulated using efficient data structures such as StringBuffer.

The editor allows users to perform operations like adding, inserting, deleting, replacing, reversing, and clearing text. The system is menu-driven, making it easy to use and understand.
This project demonstrates important OOP principles like encapsulation, class design, and method implementation. It provides a basic understanding of how real-world text editors work internally without focusing on graphical interface**.**

INTRODUCTION
: A text editor is a software application used to create and edit text. In this project, a simple text editor is implemented using Java.
The program uses StringBuffer to store text and provides multiple methods to modify it.
Users can interact with the system using a menu-driven interface.
This project focuses on backend logic and helps in understanding how text editing operations are performed in real applications.

🎯 **OBJECTIVES**

- To develop a simple text editor
- To perform text operations (add, delete, replace)
- To understand StringBuffer
- To apply OOP concepts
- To improve problem-solving skills

SCOPE
: This project is useful for beginners learning Java and OOP. It can be extended into a full application by adding GUI, file handling, and database support.

💻 **SYSTEM REQUIREMENTS**

- Hardware:

Minimum 4GB RAM
Any basic computer system

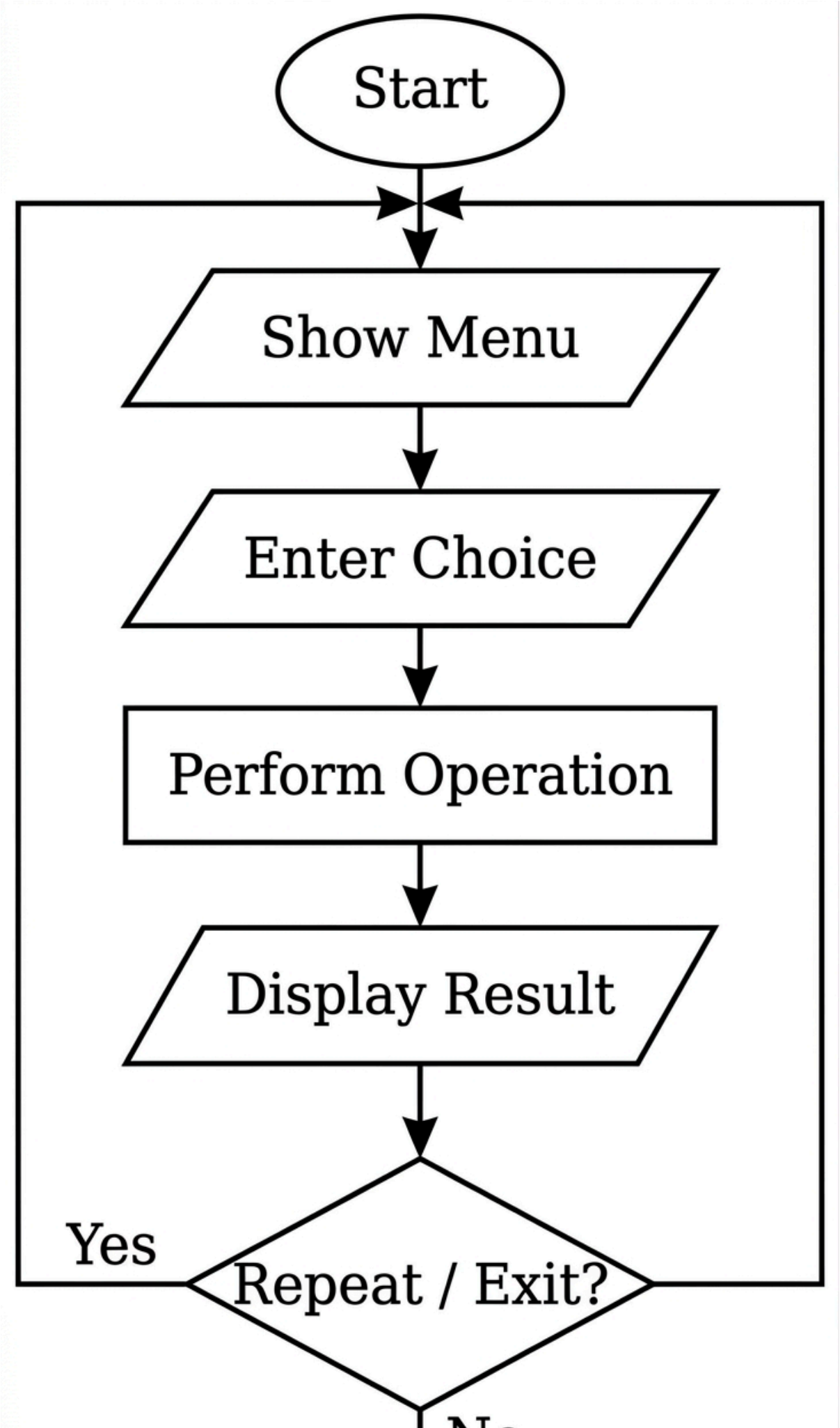
- Software:

Java JDK
IDE (VS Code / Eclipse)

⚙️ **METHODOLOGY**
1.User selects an option from menu
2.Input is taken using Scanner

3.Operation is performed on StringBuffer
4.Text is updated
5.Output is displayed

📊 FLOWCHART


Start
Show Menu
Enter Choice
Perform Operation
Display Result
Yes
Repeat / Exit?

Document Class:
Variable: StringBuffer text
Methods: addText(), insertText(), deleteText(), replaceText(), reverseText(), clearText(), displayText()
Main Class:
Method: main()
Handles user interaction

IMPLEMENTATION
The project is implemented using two main classes:
1. Document Class
This class is responsible for managing the text. It uses a private StringBuffer variable to store the text. Various methods are defined to perform operations such as adding, inserting, deleting, replacing, reversing, and clearing text.
Encapsulation is applied by keeping the text variable private and accessing it through public methods.
2. Main Class
This class contains the main method and handles user interaction. It uses the Scanner class to take input from the user and provides a menu-driven interface.
Based on user input, appropriate methods of the Document class are called to perform operations.

🖥️ **SAMPLE** INPUT / OUTPUT
Example:

1 → Add Text
Enter text: i love you

5 → Reverse Text

7 → Display Text

Output:
uoy evol i

2:34 10%

programiz.com/java-progra

Run

```java
System.out.print("Enter your
    choice: ");
choice = sc.nextInt();
sc.nextLine(); // clears leftover
    input

switch (choice) {

    case 1:
        System.out.print("Enter
            text: ");
        String add = sc.nextLine
            ();
        doc.addText(add);
        break;

    case 2:
        System.out.print("Enter
            position: ");
        int pos = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter
            text: ");
        String insert = sc
            .nextLine();
        doc.insertText(pos, insert
            );
        break;

    case 3:
        System.out.print("Enter
            start index: ");
        int start = sc.nextInt();
        System.out.print("Enter
            end index: ");
        int end = sc.nextInt();
        doc.deleteText(start, end
            );
        break;
```

Output

```
Welcome to Simple Text Editor

------ MENU ------
1. Add Text
2. Insert Text
3. Delete Text
4. Replace Text
5. Reverse Text
6. Clear Text
7. Display Text
8. Show Length
9. Exit
Enter your choice: 1
Enter text: I love you

------ MENU ------
1. Add Text
2. Insert Text
3. Delete Text
4. Replace Text
5. Reverse Text
6. Clear Text
7. Display Text
8. Show Length
9. Exit
Enter your choice: 5
Text reversed successfully.

------ MENU ------
1. Add Text
2. Insert Text
3. Delete Text
4. Replace Text
5. Reverse Text
6. Clear Text
7. Display Text
8. Show Length
9. Exit
Enter your choice: 7

Current Text: uoy evol I
```

FUTURE ENHANCEMENTS:

- Implement search functionality
- Allow deletion using words instead of indexes
- Add undo and redo features
- Convert console program into GUI application
- Add file saving and loading features

📊 GANTT CHART

Task	Duration
Planning	1 day
Designing	1 day
Coding	2 days
Testing	1 day
Finalization	1 day

📌 CONCLUSION

:The Simple Text Editor project successfully demonstrates the application of Java and Object-Oriented Programming concepts. It provides a clear understanding of how text can be managed and manipulated efficiently using StringBuffer.
The project also highlights the importance of modular programming and user interaction. It serves as a strong foundation for developing more advanced applications in the future.

📚 REFERENCES

[1]. Oracle, "Java Documentation"
https://docs.oracle.com
[2]. H. Schildt, Java: The Complete Reference